

Building Resilient Distributed Systems

Parallel and Distributed Computing - Assignment 2

Mutee ur Rehman - BSAI23005

Part 1: Analyze the Mess

StudySync is a normal FastAPI + React MVP: frontend requests come to the FastAPI backend, the backend writes to a database, receives Clerk webhooks, and calls an external LLM API. The stack works for a demo, but it fails when network timing, retries, and concurrent users become real.

Problem 1 - Synchronization

The lost update happens in the document API lifecycle between the first read and the later write. User A and User B both load document version N. User A saves first, so the database now contains A's content. User B then saves using the old copy, and the API runs a normal `UPDATE documents SET content = ? WHERE id = ?`. Since the query does not check a version number or lock the row, the database accepts B's stale write and silently replaces A's work. The root cause is missing concurrency control at both the API and database layer.

Problem 2 - Coordination

Clerk is the source of truth for subscription status, while the StudySync database stores a local copy such as `is_premium`. In the naive design, a Clerk cancellation webhook is processed once by the backend. If the network drops the POST, the app restarts, or the DB is temporarily unavailable, our side may never record the event. Because there is no durable inbox, no idempotency table, no retry worker, and no reconciliation job, the local database can remain premium forever even though Clerk says the user cancelled.

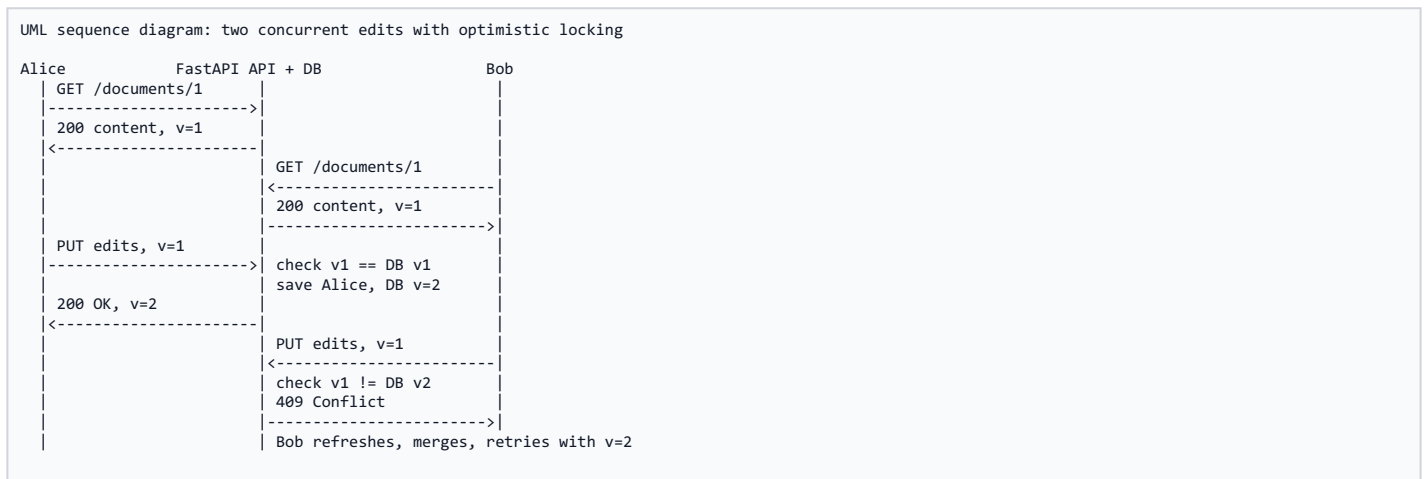
Problem 3 - Fault Tolerance

The LLM call is treated like a normal local function even though it is a remote dependency. When the LLM slows down, the FastAPI handler waits for the HTTP call until a long timeout expires. In a small MVP deployment this can tie up the available worker capacity and make unrelated users wait too. The external LLM becomes a single point of failure because there is no short timeout, no failure counter, no fallback response, and no circuit breaker.

Part 2: Design a Better System

Sync Fix - Optimistic Locking

I would add a `version` column to every document. The GET endpoint returns the current version with the content. The PUT endpoint requires the client to send that version back. The server updates only if `request.version == document.version`. If the value is stale, the API returns `409 Conflict`, so the user refreshes and merges instead of overwriting another person's work.



Coordination Fix - Durable Webhook Handling

- **Verify and store first:** verify the Clerk/Svix signature, then write the event id, type, and raw payload to a `webhook_events` table before doing business logic.
- **Idempotency:** use the Clerk/Svix event id as the idempotency key. If the same webhook arrives twice, return 200 and do not apply the cancellation twice.
- **Retry worker:** process events from the durable table or queue with exponential backoff. Failed jobs stay pending instead of disappearing.
- **Dead-letter queue:** after too many failures, move the event to a dead-letter table for manual inspection instead of dropping it.
- **Reconciliation:** run a scheduled job that asks Clerk for the current subscription state and fixes any drift in our database.

Fault Tolerance Fix - Circuit Breaker with Fallback

I implemented this one in code. The protected endpoint calls the LLM through a circuit breaker. In the closed state, requests pass through normally. After 3 consecutive failures, the breaker opens and returns a fallback answer immediately without calling the LLM. After 15 seconds, it moves to half-open and allows one test request. If that request succeeds, the breaker closes; if it fails, the breaker opens again. I also use a 5 second HTTP timeout so the app fails fast instead of waiting around 60 seconds.

State	Behavior	Why it helps
Closed	Call the LLM and count failures.	Normal users still get real LLM answers.
Open	Skip the LLM and return fallback instantly.	The failing dependency cannot hang the app.
Half-open	Allow one trial request after timeout.	The system can recover automatically.

CAP Trade-offs

Area	Choice	Explanation
------	--------	-------------

Document sync	Consistency over availability	A stale write is rejected with 409. This means one user may need to retry, but no edit is silently lost.
Webhooks	Availability now, eventual consistency later	The app accepts and stores events quickly, then retries until the local DB matches Clerk.
LLM calls	Availability and low latency over perfect answer freshness	When the LLM is down, a fallback answer is less accurate than a live LLM answer, but the app stays usable.

Since network partitions and remote failures are unavoidable, I am not trying to guarantee everything at once. For documents, correctness is more important than accepting every write. For webhooks, temporary inconsistency is acceptable as long as retries and reconciliation eventually fix it. For the LLM, user-facing availability and latency are more important than always waiting for the perfect generated answer.